

37. Les couleurs : palettes, variétés et thèmes

Une tuile porte des numéros ; la palette de sa case leur donne leurs couleurs — la même tuile peut être de trois couleurs à la fois.

1. Ce qu'il faut comprendre

La règle de la console : un carré de 8×8 — une tuile — n'a que **4 couleurs**. Pas une de plus, même sur Game Boy Color : chaque carré prend **une palette** de 4 couleurs. Le carré d'à côté peut en prendre une autre : c'est ainsi qu'un écran entier affiche bien plus de 4 couleurs.

Le jeu a 8 palettes, numérotées de 0 à 7, et la **0 est la normale** (les verts de la Game Boy). Une tuile ne contient pas de couleurs : elle contient des **numéros**, de 0 à 3. C'est la palette de la case où on la pose qui dit de quelle couleur est chaque numéro.

Le programme le montre : la même tuile `BRIQUE`, posée trois fois. `teindre(colonne, ligne, palette)` donne sa palette à une case : la palette 1 (désert), la palette 2 (océan), et la dernière n'a pas de `teindre` — elle reste dans la palette 0, la normale. **Une seule tuile, trois couleurs** : c'est tout l'intérêt des palettes.

`couleurFond(palette, numéro, rouge, vert, bleu)` choisit une couleur : chaque composante va de 0 à 31. Chaque palette va du plus **clair** (numéro 0) au plus **sombre** (numéro 3) — le jeu garde ainsi une version lisible en quatre nuances.

Dans l'atelier, sans écrire de code : les 8 palettes sont en haut de l'atelier des tuiles et de la carte. Clique une couleur, peins : ce que tu peins **prend sa palette**. «  Changer cette couleur » change une couleur **pour ce dessin seulement** : si d'autres tuiles (ou le texte) partagent sa palette, l'atelier lui en donne une copie dans une palette libre. «  Variétés » met dans une palette l'une des 11 palettes toutes prêtes (0 normale, campagne, bonbon, plage, forêt, glace, volcan, nuit, désert, océan, automne).

«  Thèmes » change d'un coup les 8 palettes du décor **et** les 8 des personnages, avec des couleurs **qui vont ensemble** : forêt, désert, glace, volcan, océan, hanté, bonbon — ou 0, la normale. Elles sont calculées : chaque palette part d'une teinte, et toutes descendent du clair au sombre par les mêmes quatre marches de lumière. **Ctrl+Z** revient en arrière.

Les personnages ont leurs 8 palettes à eux, de **3 couleurs + le transparent** (le numéro 0 ne s'affiche jamais : le décor se voit au travers). Un personnage de 16×16 est fait de 4 carrés : chacun peut avoir sa palette — jusqu'à 12 couleurs.

À toi : ajoute une quatrième brique en (10, 8) avec `teindre(10, 8, 1);` — elle prend les couleurs du désert. Puis, dans l'atelier, essaie «  Thèmes » et regarde les trois briques changer ensemble.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <couleurFond>` → [0.88.3. La fonction couleurFond\(\) — une couleur du décor](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <teindre>` → [0.89.1. La fonction teindre\(\) — une case dans une palette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **615 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage – la mise en route de la console
main()	178 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
CopierTuiles	85 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Tuiles	192 o	← CopierTuiles – 11 lettres sur 43, et 1 tuile dessinée
"UNE TUILE"	9 o	← texte() – un texte gravé
"TROIS PALETTES"	14 o	← texte() – un texte gravé

Lettres gravées : AEILNOPRSTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

couleurFond()	× 8	96 o sur place, dans main()
teindre()	× 2	34 o sur place, dans main()
texte()	× 2	22 o sur place, dans main() + EcrireTexte (10 o)
poser()	× 3	18 o sur place, dans main()
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
/* Une seule tuile : quatre NUMÉROS de couleur, de 0 (clair) à 3 (sombre). */
```

```
#include <Tuile>          // un dessin de 8 × 8 pixels
#include <couleurFond>     // choisit une couleur d'une palette du fond
#include <texte>           // écrit un texte à l'écran
#include <poser>           // pose une tuile sur une case du fond
#include <teindre>         // met une case du fond dans une palette
```

2. UN DESSIN — BRIQUE

```
Tuile BRIQUE = {
    "33333333",
    "21112111",
    "21112111",
    "33333333",
    "11121112",
    "11121112",
    "33333333",
    "00000000",
};
```



à l'écran

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    /* La palette 1 : le désert – du plus clair au plus sombre. */
    couleurFond(1, 0, 31, 29, 22);
    couleurFond(1, 1, 28, 22, 10);
    couleurFond(1, 2, 20, 12, 4);
    couleurFond(1, 3, 8, 4, 2);
    /* La palette 2 : l'océan. */
    couleurFond(2, 0, 26, 31, 31);
    couleurFond(2, 1, 8, 24, 30);
    couleurFond(2, 2, 2, 12, 24);
    couleurFond(2, 3, 0, 3, 10);

    texte(3, 3, "UNE TUILE");
    texte(3, 4, "TROIS PALETTES");

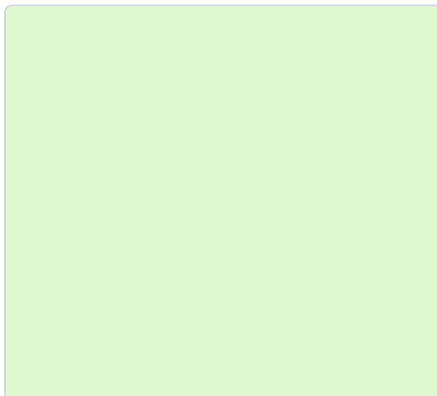
    poser(4, 8, BRIQUE);
    teindre(4, 8, 1); // la case (4, 8) prend la palette 1 : le désert
    poser(6, 8, BRIQUE);
    teindre(6, 8, 2); // la même tuile, dans la palette 2 : l'océan
    poser(8, 8, BRIQUE); // pas de teindre : la palette 0, la normale

    while (true) {
        image();
    }

    return 0;
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Trois fois la même brique : couleur sable (palette 1), bleue (palette 2), et verte (palette 0, la normale).

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d’être joués, et voici ce qu’ils ont mesuré.

- ✓ la cartouche tourne en couleur
- ✓ la même tuile est posée trois fois
- ✓ la première case a la palette 1 (1)
- ✓ la deuxième la palette 2 (2)
- ✓ la troisième, sans teindre, la palette 0 (0)
- ✓ la palette 1 porte bien le sable du désert

6. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C’est le même texte, écrit autrement.

